

v0.8.11

HTML Graph Cache Mode Integration

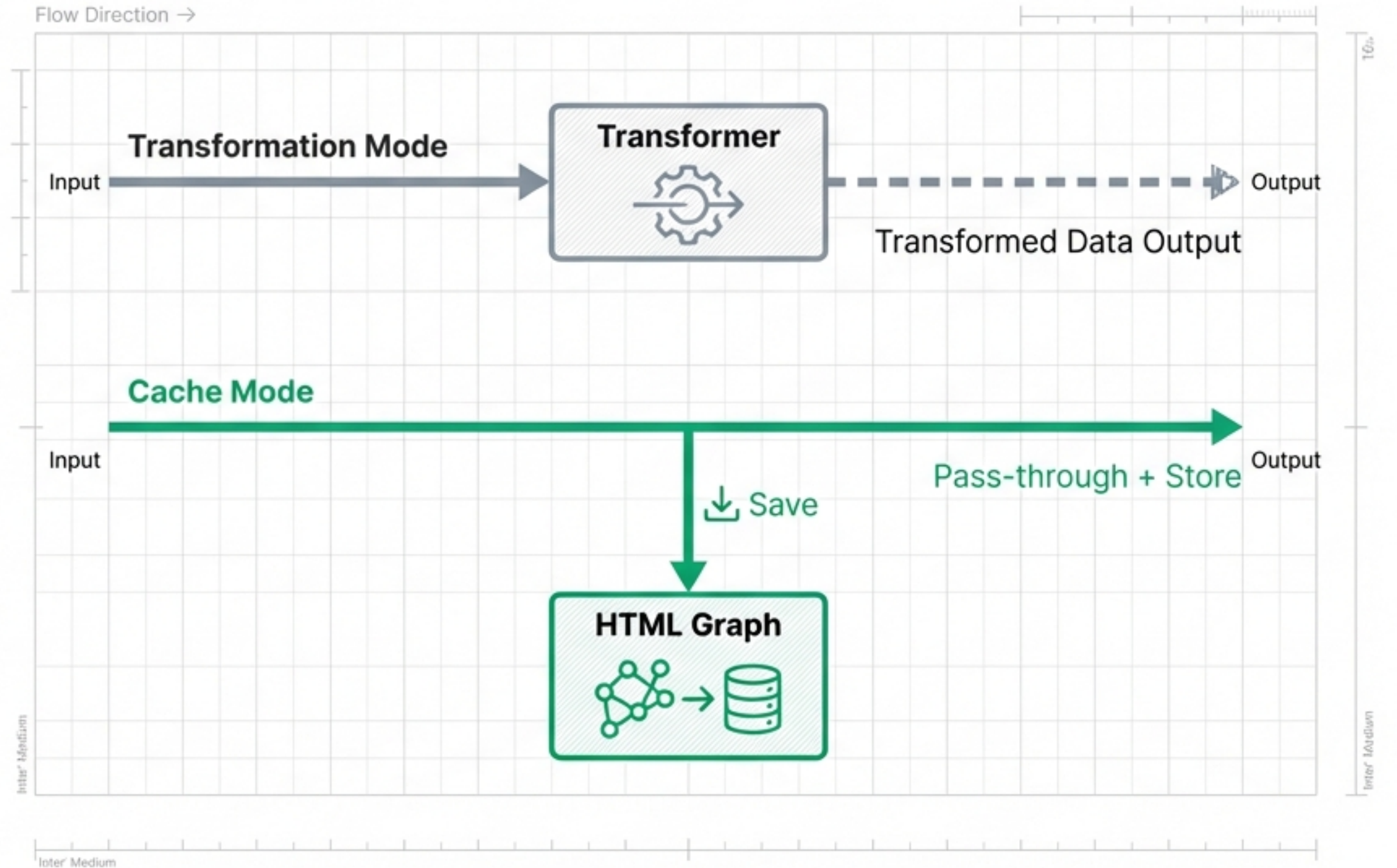
Technical Integration Guide

Target System: MitmProxy Service
Capability: Passive HTML Caching
Scope: Architecture to Validation

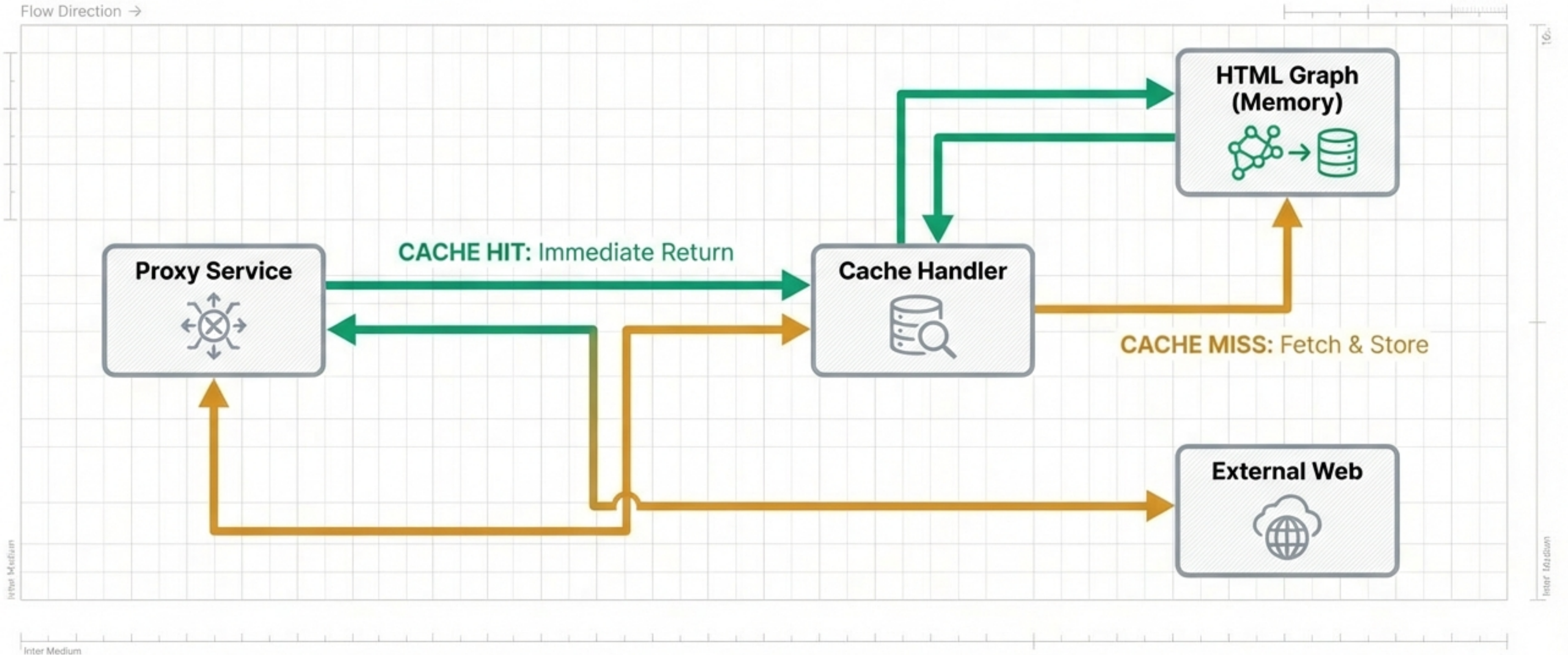
Capability Overview: Passive HTML Caching

This integration introduces the 'mitm-mode=cache' capability for the MitmProxy service. It enables passive HTML caching via the HTML Graph without triggering any content transformation.

This mode is designed purely for efficiency and persistence, operating distinctly from existing sentiment or hash transformation modes.



Architecture and Data Flow



Implementation Roadmap: Scope of Work

Files to Create

- ✓ `service/html_graph/HTML_Graph__Cache__Handler.py`
Annotate: Main Logic
- ✓ `tests/unit/service/html_graph/test_HTML_Graph__Cache__Handler.py`
Annotate: Unit Tests

Files to Modify

- 📎 `schemas/html/Enum__HTML__Transformation_Mode.py`
Annotate: Config Layer
- 📎 `service/proxy/request/Proxy__Request__Service.py`
Annotate: Integration Point A
- 📎 `service/proxy/response/Proxy__Response__Service.py`
Annotate: Integration Point B

Step 1: Configuration Layer

Enabling the System State

Before logic implementation, the system must recognize the new operational state. We update the transformation enumeration to include CACHE.

schemas/html/Enum__HTML__Transformation_Mode.py

```
class Enum__HTML__Transformation_Mode(str, Enum):  
    OFF = 'off'  
    HASHES = 'hashes'  
    SENTIMENT = 'sentiment'  
    CACHE = 'cache' # <--- NEW ENTRY
```


Step 2: The Core Logic – Cache Handler

Designing the Interface for HTML Graph Interaction

New Class

HTML_Graph__Cache__Handler

service/html_graph/HTML_Graph__Cache__Handler.py

is_cache_mode(cookies)

Checks for 'mitm-mode=cache'. Returns Boolean.

check_cache(request_data)

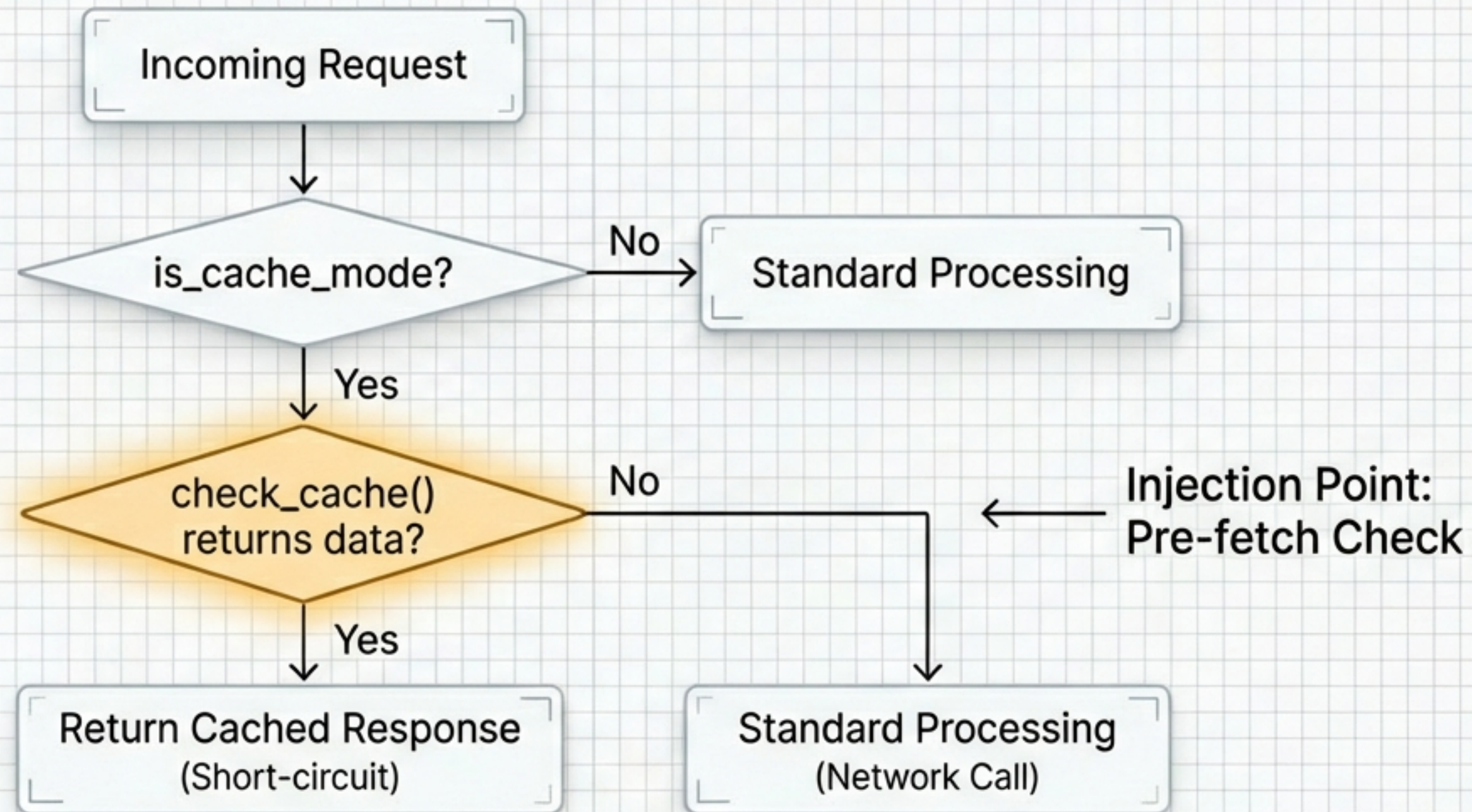
Queries HTML Graph. Returns Response Dict or None.

store_html(response_data)

Parses response. Persists to Graph. Returns Headers.

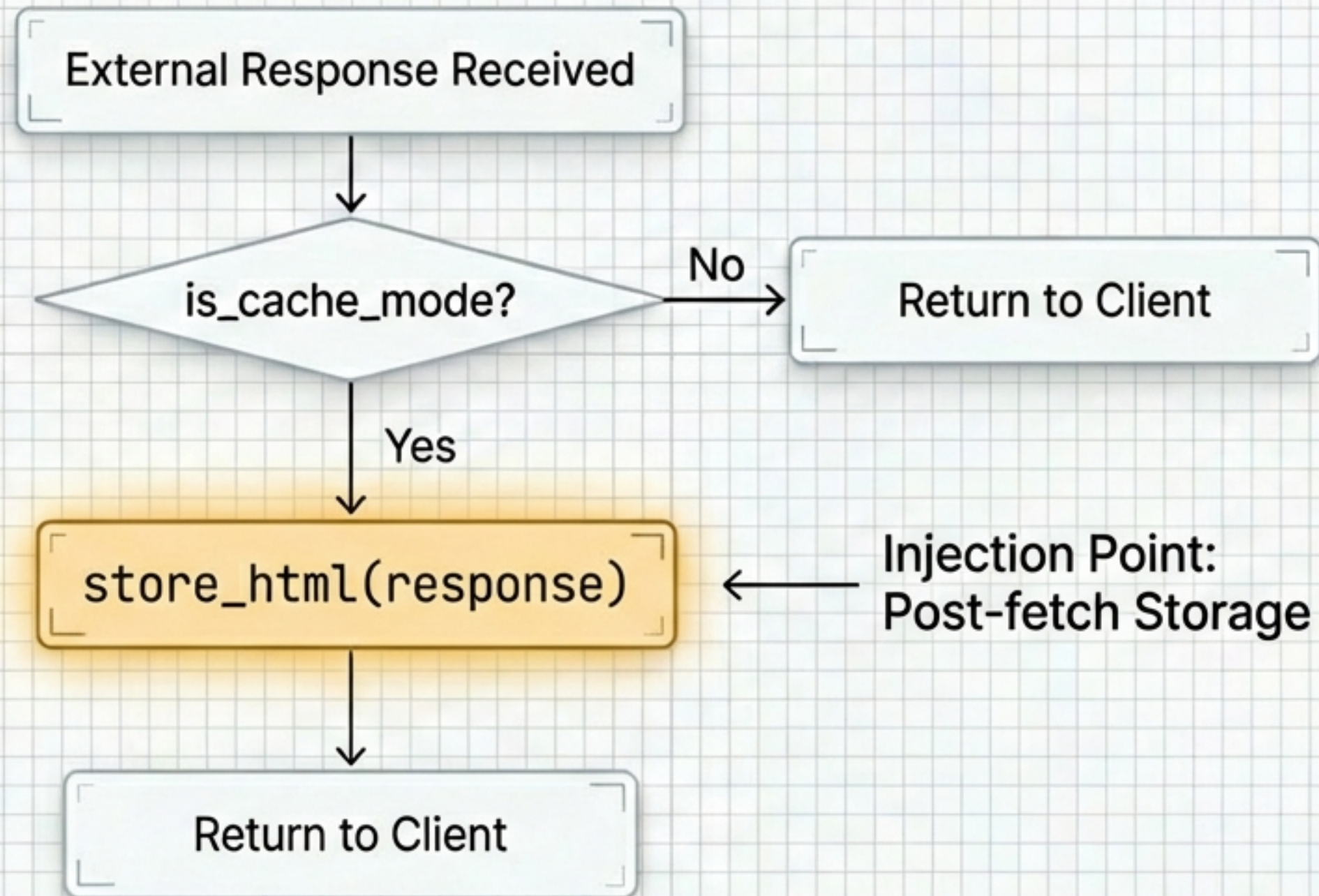
Step 3: Integration Point A – Request Interception

service/proxy/request/Proxy__Request__Service.py



Step 4: Integration Point B – Response Interception

service/proxy/response/Proxy__Response__Service.py



Ensures fresh content is captured for the next request.

Operational Logic: States and Headers



State: **CACHE HIT**

Served directly from HTML Graph.

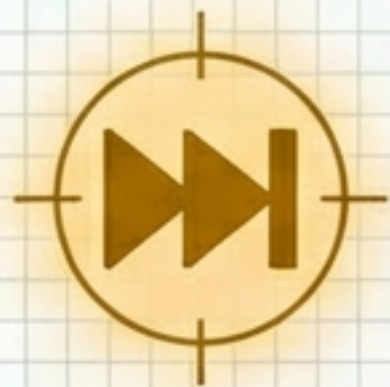
X-Cache-Status: HIT



State: **CACHE MISS**

Fetches from web, then stored.

X-Cache-Status: MISS



State: **SKIPPED**

Non-HTML content or empty response.

X-Cache-Status: SKIPPED

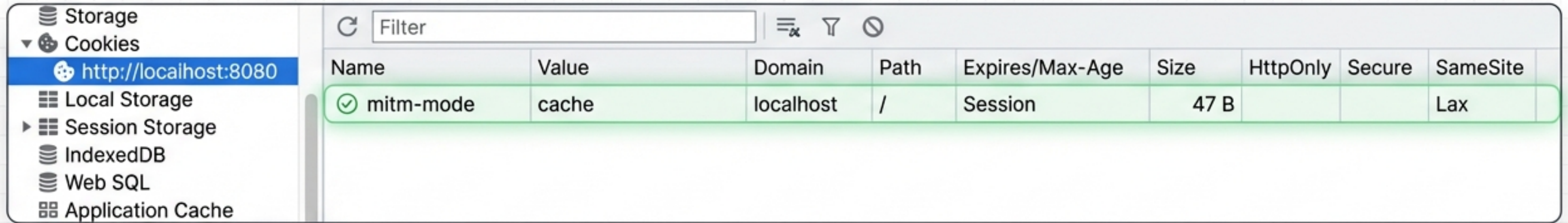
Ecosystem: Mode Coexistence

Cookie Value (mitm-mode)	Action	Transformation?
off	Pass through	No
cache	HTML Graph Cache Only	No (Raw Storage)
hashes	Replace text with IDs	Yes
sentiment	Sentiment analysis	Yes

Key Insight: The CACHE mode runs independently of the transformation pipeline. It triggers NO transformation—only raw storage.

User Controls and Browser Usage

Activating Cache Mode

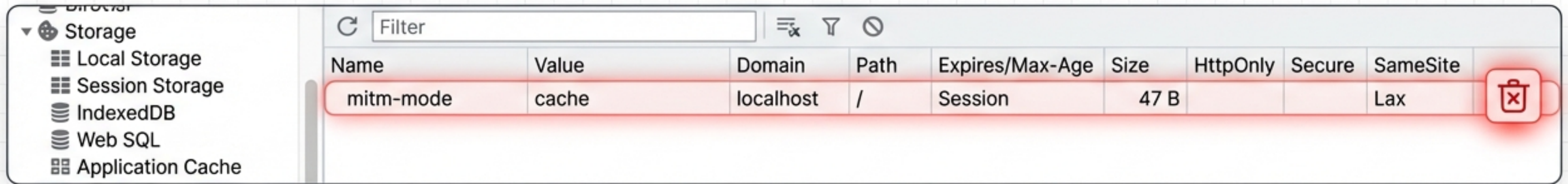


The screenshot shows the browser's developer tools with the 'Cookies' tab selected. The left sidebar lists storage types, with 'Cookies' expanded and 'http://localhost:8080' selected. The main pane displays a table of cookies. The first cookie, 'mitm-mode', has a value of 'cache' and is highlighted with a green border, indicating it is active.

Name	Value	Domain	Path	Expires/Max-Age	Size	HttpOnly	Secure	SameSite
mitm-mode	cache	localhost	/	Session	47 B			Lax

Set the cookie 'mitm-mode=cache' in the target browser.

Deactivating



The screenshot shows the same browser developer tools interface, but the 'mitm-mode' cookie is now highlighted with a red border. A red trash can icon is visible at the end of the row, indicating that the cookie is being targeted for deletion to deactivate the cache mode.

Name	Value	Domain	Path	Expires/Max-Age	Size	HttpOnly	Secure	SameSite
mitm-mode	cache	localhost	/	Session	47 B			Lax

Delete or modify the cookie to return to pass-through mode.

Quality Assurance and Testing Strategy

Robust validation using in-memory services.

- Target File:
tests/unit/service/html_graph/
test_HTML_Graph_Cache_Handler.py
- Environment: In-memory HTML
Graph Service.
- Methodology: Real service
instances (No Mocks).

```
$ pytest tests/unit/service/html_graph/  
collected 5 items  
  
test_is_cache_mode ... PASSED  
test_check_cache_hit ... PASSED  
test_check_cache_miss ... PASSED  
test_store_html ... PASSED  
test_integration_flow ... PASSED  
  
===== 5 passed in 0.45s =====
```


Deployment Checklist



Configuration: Enum updated in
'Enum__HTML_Transformation_Mode.py'.



Logic: Created 'HTML_Graph__Cache__Handler.py'.



Integration: Hooks added to Request and Response services.



Validation: All unit tests passed.

Integration Complete: Version v0.8.11 Ready for Deployment.